

Chapter 5

Lesson 25-11-2025

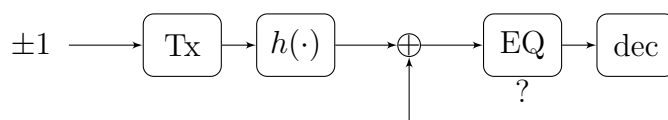
5.1 Linear channel equalization

Ref: *A signal processing perspective* (pp. 57-130), p. 31.

5.1.1 Introduction: super basic block scheme

It's a pretty old topic.

We have a transmitter, then a channel, then a noise which is added **before** anything else.



About magnetic writing; we're writing on a tape, which is noisy; we need to encode before writing.

Fractional sampling: we're sampling at multiples of the Nyquist rate; it's used in order to synchronize.

We don't need to have ± 1 , it can just be whichever input.

5.1.2 Feedback equalizer

If we have that at a certain time the receive signal is

$$y(0) = h(0)x(0) + h(1)x(-1), \quad (5.1)$$

how do we equalize this?

First of all, we'd need to estimate the channel: this means trying to understand the values of $h(0)$, $h(1)$.

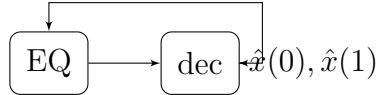
If we suppose we estimated $x(-1)$ as $\hat{x}(-1)$, what we do is implement a feedback equalizer: we'll look at

$$\frac{y(0) - h(1)\hat{x}(-1)}{h(0)} \quad (5.2)$$

which will get us $x(0)$.

We can estimate $\hat{x}(-1)$ since we generally have packets: we know the first symbols, thus we can *bootstrap* this machine.

This is the feedback equalizer:



It's also true that if we fail in decoding a symbol, this could be catastrophic: this would lead to a chain of errors.

We need to play with this type of equalization very carefully.

5.1.3 Linear zero forcing equalization (for both SISO and MIMO; only in the z domain); example 2.17

We'll see just something, but if we want to know more we can look at other books.

What we do is that we go in the z domain (see [z transform](#)); when we have a sequence, such as $h[0], h[1], h[2], \dots$, we'll write it as

$$\mathbf{h}(z) = h_0 + h_1 z^{-1} + h_2 z^{-2}. \quad (5.3)$$

We can revise this by looking at signals and systems.

A delay of n in the time domain corresponds to a multiplication with z^{-n} in the z domain.

We want to find an equalizer $w^*(z)$ such that the combination with $\mathbf{h}(z)$ is just a delay (this is the Heaviside condition):

$$w^*(z)\sqrt{G}h(z) = z^{-\Delta}. \quad (2.124)$$

The delay is a degree of freedom which can be used when designing.

We however can see that $w^*(z)$ **might not exist or it might be unstable**.

According to the zero-forcing criteria, we'd have

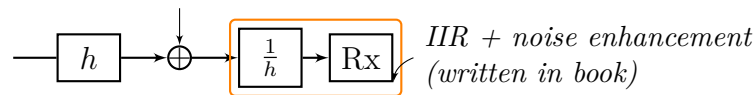
$$w^*(z) = \frac{1}{\mathbf{h}(z)} \cdot \frac{z^{-\Delta}}{\sqrt{G}}; \quad (5.4)$$

we first of all ask ourselves: **is this stable?** If it's not stable, the noise coming in might make our system explode!

The system is BIBO if the root of the polynomial $w^*(z)$ is inside the unit circle. The noise, even if bounded, might not be bounded after the system.

This is the problem of the zero forcing; the zero forcing means that we want to just have a single delay; point is that we might have just a delay but with a non-stable filter.

We then ask ourselves: **is the response $w^*(z)$ finite or infinite?** → most of the time is **infinite**



We'll then look at the zero forcing **in the MIMO case**. The interesting result is that here for many types of FIR multivariate channel we have FIR zero forcing equalizers, given that we're under certain conditions.

Let's say that the Z-transform of $H[0], \dots, H[L]$ is

$$\mathbf{H}(z) = \sum_{\ell=0}^L \mathbf{H}[\ell] z^{-\ell}; \quad (2.129)$$

this implies that the channel will become a polynomial and **we'll look for an equalizer $\mathbf{W}^*(z)$** such that

$$\mathbf{W}^*(z) \sqrt{G} \mathbf{H}(z) = \mathbf{D}(z), \quad (2.130)$$

where

- $\mathbf{D}(z)$ is a diagonal matrix containing possibly different delay terms in the form of $z^{-\Delta}$.



One thing which should be noted is that **we need to have that a left inverse exists for $\mathbf{H}$ in order for the equalizer to exist**; if we cannot have a left inverse (for various reasons, such as $\mathbf{H}$ not being full-rank), then **we won't have an equalizer**.

One condition which is considered already from this moment is that (quoting from the book):

Existence of a left FIR inverse

A MIMO transfer function $\mathbf{H}(z)$ has a left FIR inverse iff $\mathbf{H}(z)$ has full column rank for every complex $z \neq 0$.

Some other conditions will be tackled later, but just as an anticipation, they'll result in saying that the ZF equalizer exists only if

$$N_t(L_{eq} + L + 1) \leq N_r(L_{eq} + 1) \quad (2.146+)$$

What was $H[\ell]$ again?

We know that $\mathbf{H}[\ell]$ is a matrix such that

$$[\mathbf{H}[\ell]]_{i,j} = h_{i,j}[\ell]$$

is the channel's response between receiving antenna i and transmitting antenna j . From this, it will follow that

$$[\mathbf{H}(z)]_{i,j} = \mathcal{Z}[h_{i,j}[\cdot]] z.$$

“ A *signal processing perspective* (pp. 57-130), p. 33

As in the SISO case, it is in practice desirable to restrict the focus to FIR equalizers, meaning that each entry of $\mathbf{w}(z)$ corresponds to an FIR filter of order L_{eq} . Do exact left inverses of $\mathbf{H}(z)$ exist? Yes, as it turns out. More precisely, the answer is found in what is called **perfect recoverability**.

So, if the *perfect recoverability* holds, we'll be sure to find left inverses of the $\mathbf{H}$ matrix; we need the left inverse since it will be used when we'll determine the ZF equalizer.

Here in **MIMO FIR filters may exist!** Before, in SISO, **they might have not!**

About matrices: until now we saw that the inverse of whichever matrix $\mathbf{A}$ is such that $\mathbf{A}\mathbf{A}^{-1} = \mathbf{I}$ and $\mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$; this only holds if it's square; generally, the left and right inverses are different.

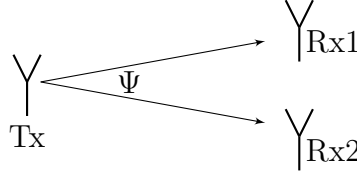
! About this paragraph

The notation we wrote here just gives an idea; everything will be treated in the time domain.

Example 2.17

For $N_t = 1$ and $N_r = 2$, find the conditions on $\mathbf{H}(z)$ such that FIR ZF equalizers exist.

If we have 1 transmit antenna and 2 receive antennas; in general we might transmit to both antennas:



| Digital always includes a delay!

We then need to decide what to do; **we might think about selecting the stronger signal**; one might have a reflection and such; this is called **antenna selection**.

It's easy to measure the amplitude of a signal, but we can't know whether the signal is clean.

Someone came up with something called **MRC (Max Ratio Combining)**: **we weight each signal with the complex conjugate**; this allows us **to have the best SNR**. This is still done, even with analog transmissions.

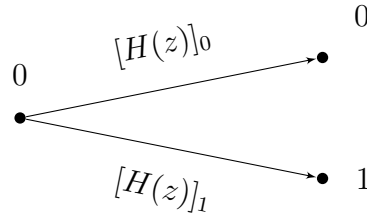
What we have is that we want to satisfy

$$[\mathbf{W}(z)]_0^* \sqrt{G} [\mathbf{H}(z)]_0 + [\mathbf{W}(z)]_1^* \sqrt{G} [\mathbf{H}(z)]_1 = z^{-\Delta}. \quad (5.5)$$

$\mathbf{H}(z)$ is a column vector:

↓ *read example (continued on next page)*

Example 2.17 (continued)



$[H(z)]_0$ might be $= h_{0,0} + h_{0,1}z^{-1} + \dots$

Anyways, we'll have

$$\mathbf{H}(z) = \begin{bmatrix} [H(z)]_0 \\ [H(z)]_1 \end{bmatrix}. \quad (5.6)$$

The only way for this vector to not be full rank is if $\mathbf{H}(z) = \mathbf{0}$ for some z .

If we have that there exists z^* such that $\mathbf{H}(z^*) = \mathbf{0}$ and this happens only if the polynomials have a common root, which is z^* ; we don't have full rank only if those two polynomials share a common root!

If those polynomials are coprime (they don't share a common root!), then it's possible to find an exact FIR inverse.

We care so much about the full rank since a left inverse only exists if $\mathbf{H}(z)$ has full column rank for every $z \neq 0$.

If we have

$$\mathbf{H}(z) = \begin{bmatrix} 1 + z^{-1} \\ 1 + z^{-1} \end{bmatrix},$$

aka the two are equal, we can't find FIR; if we have a half, then it's possible!

$$\mathbf{H}(z) = \begin{bmatrix} 1 + \frac{z^{-1}}{2} \\ 1 + z^{-1} \end{bmatrix}.$$

This is why it's not possible for SISO systems.

5.2 Moore-Penrose pseudo-inverse

In the future, we'll need another version of an inverse; this is the Moore-Penrose pseudo-inverse.

This can be applied to non-square matrices!

The pseudoinverse $\mathbf{A}^\dagger$ of an $N \times M$ matrix $\mathbf{A}$ is the unique matrix satisfying

$$\mathbf{A}\mathbf{A}^\dagger\mathbf{A} = \mathbf{A} \quad (\text{B.30})$$

$$\mathbf{A}^\dagger\mathbf{A}\mathbf{A}^\dagger = \mathbf{A}^\dagger \quad (\text{B.31})$$

and also such that $\mathbf{A}\mathbf{A}^\dagger, \mathbf{A}^\dagger\mathbf{A}$ are both **Hermitian**.

Taking the already known concept of inversion, we have that:

- If $\mathbf{A}^* \mathbf{A}$ is invertible, then

$$\mathbf{A}^\dagger = (\mathbf{A}^* \mathbf{A})^{-1} \mathbf{A}^* \quad (\text{B.32})$$

$$\mathbf{A}^\dagger \mathbf{A} = \mathbf{I}; \quad (5.7)$$

this may be the case if $N \geq M$ (tall matrix).

- If $\mathbf{A} \mathbf{A}^*$ is invertible, then

$$\mathbf{A}^\dagger = \mathbf{A}^* (\mathbf{A} \mathbf{A}^*)^{-1} \quad (\text{B.33})$$

$$\mathbf{A} \mathbf{A}^\dagger = \mathbf{I}; \quad (5.8)$$

this may be the case if $N \leq M$ (fat matrix).

- If $\mathbf{A}$ is square and invertible, then (B.32), (B.33) coincide and the pseudoinverse reduces to the regular inverse,

$$\mathbf{A}^\dagger = \mathbf{A}^{-1}.$$

⚠ The right inverse and the left inverse are not the same!

⚠ This only holds if the matrix is **full-rank** (which is then the reason why we ask that the $\mathbf{H}$ matrix is full-rank).

We'll use this a lot in the course.

Another point is that if we remember the **singular value decomposition**, then we have

$$\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^*$$

$$\mathbf{A}^\dagger = \mathbf{V} \mathbf{\Sigma}^{-1} \mathbf{U}^*.$$

By the way, we can do this in matlab using the `pinv()` function.

5.3 Actually finding the MIMO zero forcing equalizer

Let's write the problem in the time domain, going from

$$\mathbf{W}^*(z) \sqrt{G} \mathbf{H}(z) = \mathbf{D}(z), \quad (2.130)$$

where $\mathbf{D}(z)$ is made of diagonal elements:

$$\mathbf{D}(z) = \begin{bmatrix} z^{-\Delta_1} & & 0 \\ & z^{-\Delta_2} & \\ 0 & & \ddots \end{bmatrix}$$

to

$$\sqrt{G} \sum_{\ell=0}^{L_{eq}} \mathbf{W}^*[\ell] \mathbf{H}[n-\ell] = \text{diag}[\delta[n-\Delta_0], \dots, \delta[n-\Delta_{N_t-1}]]. \quad (2.132)$$

where:

- $\mathbf{W}$ is a $N_r \times N_t$ matrix (I deduced the N_t from the fact that later the columns are indexed with j).
- $\mathbf{H}$ is a $N_r \times N_t$ matrix. (Note: usually H_{ij} means from j to i).

This is a diagonal matrix with delays since we designed our equalizer to establish so.

We've allowed ourselves to have different delays ($\Delta_n, n \in [0, N_t - 1]$); finding the zero forcing equalizer means solving the following equation for the stream j :

$$\sqrt{G} \sum_{\ell=0}^{L_{eq}} \mathbf{w}_j^*[\ell] \mathbf{H}[n - \ell] = \mathbf{d}_j[n] \quad n = 0, 1, \dots, \underbrace{L_{eq} + L}_{\rightarrow \text{consequence of convolution}}; \quad (2.133)$$

It makes perfect sense to now ask ourselves what all the parameters are, specifically; let's see them in detail.

 $\mathbf{w}_j[\ell]$

$\mathbf{w}_j[\ell]$ is a $N_r \times 1$ column vector such that

$$\mathbf{w}_j[\ell] = \mathbf{W}[\ell]_{:,j} : \begin{bmatrix} w_{1,j}[\ell] \\ \vdots \\ w_{N_r,j}[\ell] \end{bmatrix}.$$

It represents the equalization applied to the signals transmitted by the antenna j towards all the receiving antennas.

 $\mathbf{d}_j[n]$

With reference to the equation (2.133) we define $\mathbf{d}_j[n]$ as the **target delays for transmitted signal** j , with $j = 0, \dots, N_t - 1$; it will be a $N_t \times 1$ vector made of **all zeros** and a $\delta[n - \Delta_j]$ **at position** j , where $\Delta_j \in [0, \dots, L_{eq} + L]$ is the target delay for the transmitted signal j .

We can formulate (2.133) in vector notation as

$$\sqrt{G} \bar{\mathbf{w}}_j^* \mathcal{T} = \bar{\mathbf{d}}_j, \quad (2.137)$$

with the various elements being as described in the following:



$\bar{\mathbf{w}}_j$

$\bar{\mathbf{w}}_j$ is a $N_r(L_{eq} - 1) \times 1$ vector obtained by stacking all $\mathbf{w}_j$ vectors on top of one another, as

$$\bar{\mathbf{w}}_j = \begin{bmatrix} \mathbf{w}_j[0] \\ \vdots \\ \mathbf{w}_j[L_{eq}] \end{bmatrix}, \quad (2.134)$$

embedding all the equalizer responses over the delay dimension, always referring to transmit antenna j .



$\bar{\mathbf{d}}_j$

We then define $\bar{\mathbf{d}}_j$ as a $N_t(L_{eq} + L + 1) \times 1$ vector:

$$\bar{\mathbf{d}}_j = \begin{bmatrix} \mathbf{d}_j[0] \\ \vdots \\ \mathbf{d}_j[L_{eq} + L] \end{bmatrix}; \quad (2.135)$$

discussing the positions at which this vector is non-zero deserves its own discussion. Due to how $\mathbf{d}_j[n]$ is defined, we know that it will have a 1 only at position j and only if $n = \Delta_j$. Let's now consider $\bar{\mathbf{d}}_j$ for a simple case such as $N_t = 2, j = 0, L_{eq} + L = 1$; we'll have

$$\bar{\mathbf{d}}_0 = \begin{bmatrix} \mathbf{d}_0[0] \\ \mathbf{d}_0[1] \end{bmatrix} = \begin{bmatrix} \begin{bmatrix} \delta[0 - \Delta_0] \\ 0 \end{bmatrix} \\ \begin{bmatrix} \delta[1 - \Delta_1] \\ 0 \end{bmatrix} \end{bmatrix};$$

in this simple example we see that the various subvectors are located at multiples of N_t , specifically at $nN_t, n \in [0, L_{eq} + L]$; in each subvector, the δ is located at the j -th element. We'll thus have the various deltas at positions

$$N_t n + j, \quad n = 0, \dots, L_{eq} + L$$

where j is fixed by the vector $\bar{\mathbf{d}}_j$ we're considering.

It's also true that the δ s will only be 1 if $n = \Delta_j$ and we can thus simplify the previous equation by saying that we'll have 1 in the vector at positions

$$N_t \Delta_j + j,$$

which of course can only be satisfied for those Δ_j which are $\Delta_j \leq L_{eq} + L$.

Toeplitz matrix $\bar{\mathcal{T}}$

The Toeplitz matrix $\bar{\mathcal{T}}$ is defined as

$$\bar{\mathcal{T}} = \begin{bmatrix} \mathbf{H}[0] & \dots & \mathbf{H}[L] & \mathbf{0} & \dots & \mathbf{0} \\ \mathbf{0} & \mathbf{H}[0] & \dots & \mathbf{H}[L] & \mathbf{0} & \mathbf{0} \\ \vdots & & \ddots & & \ddots & \vdots \\ \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} & \mathbf{H}[0] & \dots \mathbf{H}[L] \end{bmatrix}, \quad (2.136)$$

having dimensions of $N_r(L_{eq} + 1) \times N_t(L_{eq} + L + 1)$.

We reach a linear system of equations $\rightarrow$ under determined means infinite sol, over determined means no-sol.

We then consider that we're trying to minimize the error in (2.137), which is proportional to

$$\|\sqrt{G}\bar{\mathbf{w}}_j^* \bar{\mathcal{T}} - \bar{\mathbf{d}}_j^*\|^2. \quad (RIP \text{ Tomba}^\dagger) \quad (2.138\text{-ish})$$

We can show (*A signal processing perspective* (pp. 57-130), p. 35) that (2.138-ish) can be minimized using the left Moore-Penrose pseudo-inverse of the matrix $\bar{\mathcal{T}}^*$, aka $(\bar{\mathcal{T}}^*)^\dagger$:

$$\bar{\mathbf{w}}_j^{ZF} = \frac{1}{\sqrt{G}}(\bar{\mathcal{T}}^*)^\dagger \bar{\mathbf{d}}_j \quad \text{time domain} \quad (2.145)$$

$$\mathbf{W}^{ZF} = \frac{1}{\sqrt{G}}(\bar{\mathcal{T}}^*)^\dagger \bar{\mathbf{D}} \quad \text{z domain} \quad (2.146)$$

We can see how we can find the zero-forcing solution for MIMO.

I'd like to elaborate a lil bit on this; we essentially have that

$$\sqrt{G}\mathbf{W}^{ZF*} \mathcal{T} = \mathbf{D}^* \implies \mathbf{W}^{ZF*} = \frac{1}{\sqrt{G}}\mathbf{D}^*(\mathcal{T})^\dagger.$$

Now we need to do the complex transpose of that, in order to find the equalizer.

Before doing that, it's relevant to know that

$$(\bar{\mathcal{T}}^\dagger)^* \stackrel{\text{tall}}{=} [(\mathbf{A}^* \mathbf{A})^{-1} \mathbf{A}^*]^\dagger = \mathbf{A}(\mathbf{A}^* \mathbf{A})^{-1} = (\bar{\mathcal{T}}^*)^\dagger;$$

then we see that the complex transpose of the equation up there does indeed give (2.146).

z domain?

I think it was professor to write "z domain"; I'm positive this whole thing goes for the time domain!

For a frequency flat channel, this simplifies to

$$\mathbf{W}^{ZF} = \frac{1}{\sqrt{G}}(\mathbf{H}^\dagger)^*.$$

Just as a sidenote, this equation gives what some exercises refer to as the *perfect equalizer*.

One important thing which was overlooked (until here) is that this comes as a solution to (2.137); thing is that **said solution only exists if**

$$N_t(L_{eq} + L + 1) \leq N_r(L_{eq} + 1) \quad (2.146+)$$

(aka **only exists if $\bar{\mathcal{T}}$ is either square or tall**).



The $\mathbf{W}^{ZF}$ matrix is defined as

$$\mathbf{W}^{ZF} = [\bar{\mathbf{w}}_0^{ZF} \quad \dots \quad \bar{\mathbf{w}}_{N_t-1}^{ZF}],$$

in a way such that the j -th column represents the equalizer coefficients for the j th transmitted signal in a way such that the values from kN_r to $(k+1)N_r, k \in \mathbb{Z}$ will represent the coefficients used to equalize at the k th receiver.



Matrix $\bar{\mathbf{D}}$ is defined as

$$\bar{\mathbf{D}} = [\bar{\mathbf{d}}_0 \quad \dots \quad \bar{\mathbf{d}}_{N_t-1}],$$

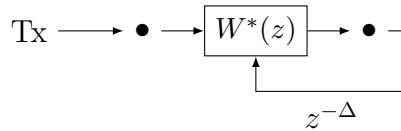
which, due to what has been said before, will be zero except for positions

$$(N_t\Delta_j + j, j),$$

which will be 1, if the position exists in the matrix.

5.4 MIMO Degrees of Freedom and ZF Equalization

Let's conceptually **look at a problem which arises only when we look at SISO** due to the lack of degrees of freedom. If we have 3 transmitter and 4 receivers, we can introduce delays to make everything look like just one delay. If we just have 1 receiver, we can't make everything look just like one delay, though:



For the first antenna, we modify $W^*(z)$ so that it can transform the first channel response into a delay $z^{-\Delta}$; the other channel response will not be such that the same $W^*(z)$ will be able to enforce the same delay as a result (at least if the response is different, which is generally the case): we've already used the only degree of freedom we had by setting Δ .

Rem. In the MIMO case, we have more degrees of freedom and that's why we generally have a solution, provided that $\mathbf{H}$ is full rank.

Example 2.18

Compute the ZF equalizer for a frequency-flat channel with $N_r \geq N_t$.

In a frequency flat channel we can see that the matrix is just a pseudo inverse of $\mathbf{H}$, since we have $T = \mathbf{H} \rightarrow$ delay of L . Then do not $L_{eq} \rightarrow \infty$ (minimize eq. error). We'll thus have:

$$W^{ZF} = \frac{1}{\sqrt{C}} \mathbf{H}^+ \quad (5.9)$$

There's a difference between the SISO and the MIMO cases.

- **SISO:** we have the **problem of stability** and even for simple channels we might **not have a causal filter**.
- **MIMO:** The **perfect equalizer** of a **FIR channel** is **FIR**.

Example 2.19

Verify that the W^{ZF} derived in Example 2.18 effects perfect ZF equalization.

Solution

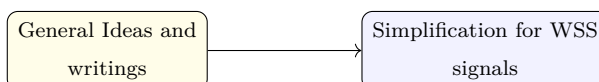
The application of the equalizer to the channel gives:

$$W^{ZF} \sqrt{C} \mathbf{H} = (\mathbf{H} \mathbf{H}^+)^{-1} \mathbf{H}^H \mathbf{H} \Rightarrow \mathbf{I} \quad \text{Identity} \quad (5.10)$$

If we ignore the noise, we have that the channel behaves like an Identity.

If we have 4 antennas in transmission and reception, it's like if we had 4 parallel channels: this is a big step forward.

5.5 LMMSE equalization



Until now **we ignored noise**: the big problem with zero forcing which does not disappear with MIMO is that **noise generally gets amplified**.

If we multiply the w with y , we get an error on top of the delay. We'll introduce a new concept, aka **trying to minimize the noise**. We're not just inverting the channel: we now have another target; we're trying to invert the channel as much as we can while also limiting the noise as much as we can.

ZF equalization neglects the noise: we instead would like to **take the noise into account**; in order to do so, we use the MMSE metric we already introduced. Our equalizer

will then be a MMSE estimator and since we stay in the context of linear equalization, it will be a Linear MMSE estimator (LMMSE).

In this section we'll try to derive a LMMSE estimator using the theoretical concept we defined for MMSE estimation.

For this derivation we consider both $\mathbf{x}[n], \mathbf{s}[n]$ as **wide sense stationary**, using the notations:

$$R_x[l] = E[\mathbf{x}[n]\mathbf{x}^*[n+l]] \quad (5.11)$$

$$R_x[l] = N_0\delta[l] \cdot I \quad (5.12)$$

(Pass on this / get to some)

It can be shown that thanks to the transmit-receive relationships we defined:

$$R_y[l] = G \sum_{i=0}^L \sum_{j=0}^L \mathbf{H}[i] R_x[l+i-j] \mathbf{H}^*[j] + N_0\delta[l]I \quad (5.13)$$

$$R_{yx}[l] = \sqrt{G} \sum_{i=0}^L \mathbf{H}[i] R_x[l+i] \quad (5.14)$$

We want to look at the difference between $\mathbf{x}[n - \Delta]$ (which is the desired signal) and the equalized output (obtained by convolving the output y ($N_r \times 1$) and the equalizer W ($N_t \times N_r$)). We need to remember that y will have the noise in it:

$$\hat{x}[n] - x[n - \Delta] = \sum_{l=0}^{L_{eq}} \mathbf{W}^*[l] \mathbf{y}[n - l] \quad (5.15)$$

⚠ Totally not trivial implication

Differently from what we have in the original treatment, here we're not considering a frequency-flat channel. Rather, this channel has time dependency: for this reason, we're considering the convolution between $\mathbf{W}$ and y .

We want to minimize this difference here! We can also obtain a matricial notation by stacking the various vectors/matrices; we'll obtain:

$$\hat{\mathbf{x}} = x[n - \Delta] - \mathbf{W}^* \vec{y}[n] \quad (5.16)$$

This is good to find the LMMSE estimator (*in the book are all the details, professor does not really care about them*).

Rem. The book never states this explicitly, but those matrices are:

- $\mathbf{W}^* : N_t \times N_r(L_{eq} + 1)$
- $\vec{y}[n] : N_r(L_{eq} + 1)$

And they're obtained through a simple stacking.

Although neither the book or professor were really clear about this, I'd say that our equalizer is designed so that

$$\mathbf{W}^* y[n] \approx x[n - \Delta] \quad (5.17)$$

The problem is that ZF equalizers amplify the noise; we'd like to have a compromise between equalization of the channel and non-amplification of the noise.

We thus form the W^{MMSE} estimator following from **The estimator is simply**

$$\mathbf{W}^{LMMSE} = \mathbf{R}_y^{-1} \mathbf{R}_{yx} \quad (5.18)$$

which in this case (that of *WSS signals/noise*) yields

$$W^{MMSE} = \mathbf{R}_{y[n]}^{-1} \mathbf{R}_{y[n]x[n-\Delta]} \quad (5.19)$$

$$= \begin{bmatrix} R_y[0] & R_y[1] & \dots & R_y[L_{eq}] \\ R_y[1] & R_y[0] & \dots & R_y[L_{eq} - 1] \\ \vdots & \vdots & \ddots & \vdots \\ R_y[L_{eq}] & R_y[L_{eq} - 1] & \dots & R_y[0] \end{bmatrix}^{-1} \begin{bmatrix} R_{yx}[-\Delta] \\ R_{yx}[1 - \Delta] \\ \vdots \\ R_{yx}[L_{eq} - \Delta] \end{bmatrix} \quad (5.20)$$

We then have that from

$$E = \mathbb{E}[(\mathbf{x} - \hat{x})\mathbf{y}^H(\mathbf{x} - \hat{x})^H] \quad (5.21)$$

$$= \mathbf{R}_x - \mathbf{R}_{yx} - \mathbf{R}_{yx}^H + \mathbf{R}_y \quad (5.22)$$

$$= \mathbf{R}_x - \mathbf{W}^H \mathbf{R}_y^{-1} \mathbf{R}_{yx} - \dots \quad (5.23)$$

we can find the MMSE matrix by considering the various matrices we've already found, which will specifically give:

$$E = \mathbb{E}[\tilde{x}[n]\tilde{x}^*[n]] \quad (5.24)$$

$$= R_x[0] - R_{yx[n]y[n]}^H R_{y[n]}^{-1} R_{yx[n]x[n-\Delta]} \quad (5.25)$$

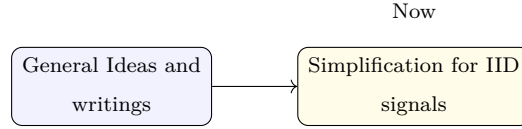
We don't need to remember the manipulations which are involved.
There won't be exercises about this!

! Exam

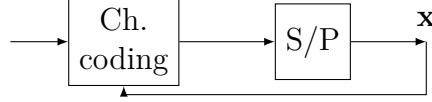
We can bring 1A4 paper, with printings on only one side. We'll be able to use them for everything but not for the quizzes!!

! Warning

NEVER forget the noise, since it's the most important thing here.



If, in order to simplify things, we write as follows:



We have that the values of $\mathbf{x}$ are actually dependent, since the channel coding introduces dependencies and redundancy.

We make an **(incorrect!!)** assumption which is that the vector $\mathbf{x}$ is made of **IID symbols**; this assumption allows us to have a simple writing of $\mathbf{R}_x$:

$$R_x[l] = \frac{E_x}{N_t} \delta[l] I \quad (5.26)$$

If we choose E_x in a specific manner, the R_x matrix becomes an identity, which simplifies things by a lot. This won't work in real systems, tho!

We'll obtain that the MMSE matrix can be written as:

$$\tilde{W}^{MMSE} = \mathbf{R}_{\tilde{y}[n]}^{-1} \mathbf{R}_{\tilde{y}[n]x[n-\Delta]} \quad (5.27)$$

$$= \frac{1}{\sqrt{G}} \left(\mathcal{T}^H \mathcal{T} + \frac{N_t}{\text{SNR}} \mathbf{I} \right)^{-1} \mathcal{T}^H D \quad (5.28)$$

$$= \frac{1}{\sqrt{G}} \mathcal{T}^H \left(\mathcal{T} \mathcal{T}^H + \frac{N_t}{\text{SNR}} \mathbf{I} \right)^{-1} D \quad (5.29)$$

The relevant thing is the SNR: if we push the SNR to $+\infty$, we have that the noise is disappearing and we'll find again the zero forcing equalizer! It makes sense that if the noise is disappearing the noise equalizer and the zero-forcing equalizer converge to be the same.

■ This will never be asked, but for completeness:

The $\mathcal{T}$ matrix is

✍ **Toeplitz matrix $\mathcal{T}$**

The Toeplitz matrix $\mathcal{T}$ is defined as

$$\mathcal{T} = \begin{bmatrix} H[0] & \dots & H[L] & 0 & \dots & 0 \\ 0 & H[0] & \dots & H[L] & \dots & 0 \\ \vdots & & \ddots & & \ddots & \vdots \\ 0 & \dots & 0 & H[0] & \dots & H[L] \end{bmatrix} \quad (5.30)$$

having dimensions of $N_r(L_{eq} + 1) \times N_t(L_{eq} + L + 1)$.

and D is all zeros except for

$$[D]_{\Delta N_t : \Delta N_t + N_t - 1, :} = \mathbf{I} \quad (5.31)$$

The alternative form is obtained using the **Matrix inversion lemma**. We know that this is a tool which is very much used.

What's important now is **the error**, which will be:

$$E = \frac{E_x}{N_t} \mathbf{I} - \frac{E_x}{N_t} D^T \mathcal{T}^* \left(\mathcal{T} \mathcal{T}^* + \frac{N_t}{\text{SNR}} \mathbf{I} \right)^{-1} \mathcal{T} D \quad (5.32)$$

✍ **Note:** EXAM TYPE OF QUESTION: demonstrate that G goes to zero if nr goes to infty

If we **push the SNR to infinity**, we again have that the whole second term becomes $(E_x/N_t)I$ and thus we'd have $E \rightarrow 0$: we're again in the zero-forcing situation.

We didn't calculate the error in case of zero forcing; we'll see that the error is actually worse than this one.

 About the inversion of a matrix of matrices

We can correctly invert a matrix by finding its determinant:

$$A = \begin{bmatrix} d_{11} & d_{12} \\ d_{21} & d_{22} \end{bmatrix} \quad (5.33)$$

$$\det(A) = d_{11}d_{22} - d_{12}d_{21} \quad (5.34)$$

Instead:

$$\mathcal{A} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \quad (5.35)$$

We'll have

$$\det(\mathcal{A}) = \det A_{11} \det A_{22} \quad (5.36)$$

$$- \det A_{12} \det A_{21} \quad (5.37)$$

We can find this as reference 125 in the book.

✗ Recommended read: example 2.21

Example 2.21, which was mentioned by professor many times, uses (2.163) together with some other smart tricks in order to obtain the LMMSE estimator for frequency flat channels, using the precoding writing of the signal,

$$x[n] = \sqrt{\frac{E_s}{N_t}} F[s] s[n] \quad (5.38)$$

, also assuming that the $s[n]$ symbols are IID, strict-sense stationary and unit-variance: this, together with the imposition that $\Delta = 0$, allows us to write that

$$\mathbf{R}_x[0] = \frac{E_s}{N_t} \mathbb{E}[\mathbf{F}[n] \mathbf{s}[n] \mathbf{s}^*[n] \mathbf{F}^*[n]] = \frac{E_s}{N_t} \mathbf{F}[n] \mathbf{F}^*[n]$$

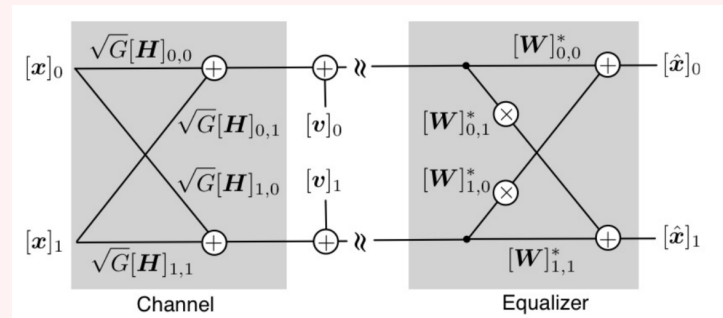
(which is admittedly a bit of a stretch, since the book just said that $\mathbf{R}_x[0] = \mathbf{F} \mathbf{F}^*$, assuming that all symbols are precoded through the same precoder and that $[0]$ expresses a delay of 0).

In turn, this eventually allows us to write

$$\mathbf{W}^{\text{MMSE}} = \frac{1}{\sqrt{G}} \left(\mathbf{H} \mathbf{F} \mathbf{F}^* \mathbf{H}^* + \frac{N_t}{\text{SNR}} \mathbf{I} \right)^{-1} \mathbf{H} \mathbf{F}, \quad (2.182)$$

for which we then have

❓ Normally asked question: MIMO Eq



What is the length of the equalizer?

$L_{eq} = 2$? 1? **No.**

L_{eq} is **zero** because we don't have memory. **We have only a matrix.**

Convention:

indices: $\boxed{0}, 1, \dots, L_{eq}$
 $\underbrace{\hspace{1.5cm}}_{L_{eq}+1}$